Лекция 06

Безопасная обработка данных

Как сделать так, чтобы входные данные оставались данными?

Факультет компьютерных наук

Бакалаврская образовательная программа 09.03.04 «Программная инженерия» · 3 курс

Дисциплина «Разработка безопасного программного обеспечения» · 2026/2027 учебный год



После лекции вы сможете

1. определить точки поступления недоверенного ввода по границам доверия
2. различить синтаксическую и семантическую валидацию и применить нормализацию
3. объяснить механизм инъекции и способ разделения данных и команд
4. разделить безопасный ответ клиенту и диагностическую запись
5. предложить негативную проверку и прослеживаемое подтверждение исправления

Материал лекции 06 входит в КР

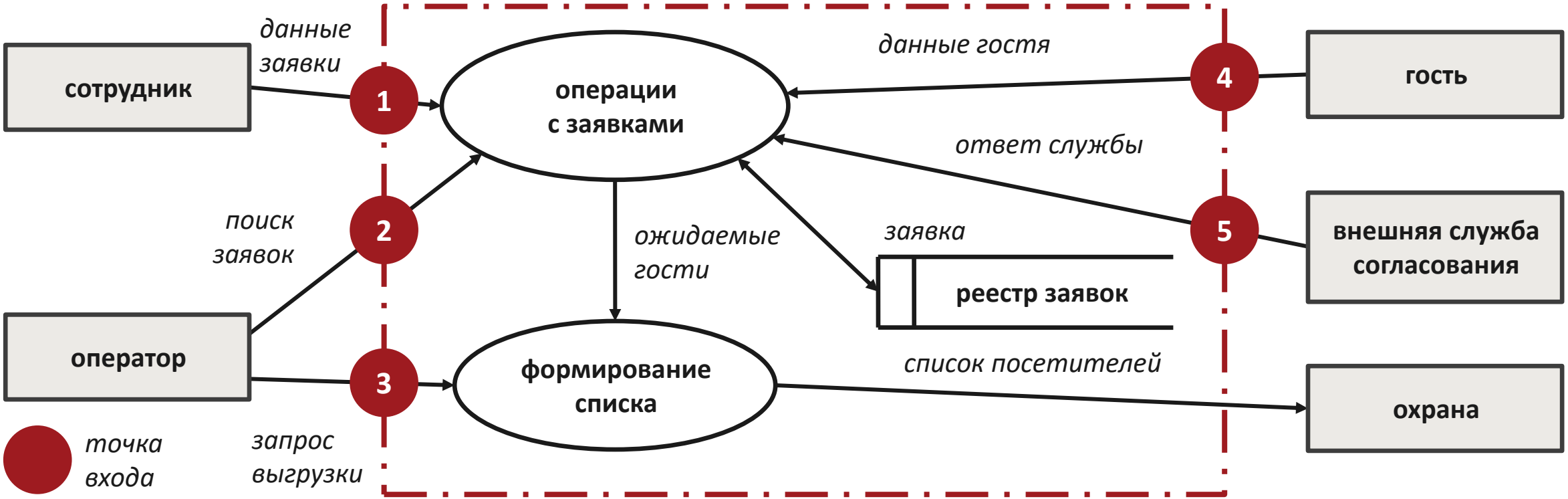
Секция 1

Недоверенный ввод и границы доверия

Модель угроз уже показывает, где данные пересекают границу доверия



Граница доверия: где приходит ввод?



Оператор вводит фамилию O'Брайен.
Почему поиск отвечает ошибкой, хотя фамилия допустима?

Граница доверия показывает, где искать ввод, но не механизм сбоя

Источников ввода больше, чем кажется

Поле формы

дата визита в заявке

Параметр API

имя гостя в запросе поиска

Заголовок запроса

язык, тип клиента, метки

Загружаемый файл

имя и содержимое вложения

Ответ внешней службы

результат согласования визита

Сохранённая запись

заявка из реестра

СОХРАНЕНО ВЧЕРА – ИСПОЛЬЗУЕТСЯ СЕГОДНЯ

прежняя проверка относилась к прежнему использованию; в новом контексте гарантии пересматриваются

От места использования зависят необходимые гарантии для значения

Клиентская проверка не защищает сервер

ФОРМА В БРАУЗЕРЕ

сегодня 12 октября 2026 года; календарь даёт выбрать только будущие дни

ПРЯМОЙ ЗАПРОС К API, МИНУЯ ФОРМУ

```
visit_date = "2026-10-01"
```

Сервер: создание заявки

ПРАВИЛО НА СЕРВЕРЕ

дата допустима для нового визита

через форму, **2026-10-20** – заявка создана

прямой запрос, **2026-10-01** – отклонён, заявка не создана

Гарантирует ли это, что сервер никогда не получит дату из прошлого?

**Сервер обеспечивает обязательное правило,
даже если интерфейс его уже проверяет**



Карта точек входа

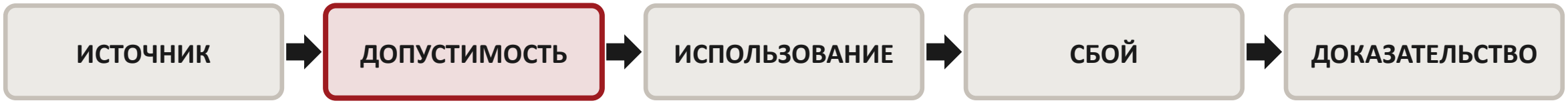
Источник, вход	Компонент-получатель	Ожидаемая форма	Предметное ограничение	Где используется	Реакция на нарушение
форма создания заявки	серверное создание заявки	дата ГГГГ-ММ-ДД	допустимая дата будущего визита	сохранение и список для охраны	отклонить; заявку не создавать
параметр `guest_name` API поиска	серверный поиск заявок	строка UTF-8	непустая, не длиннее 80 символов	фильтр поиска в хранилище	отклонить; поиск не выполнять

Карта задаёт правило приёма, дальнейшее применение и наблюдаемую реакцию на нарушение

Секция 2

Валидация и нормализация

Правильный формат не гарантирует допустимое действие



Валидация: форма и смысл

Валидация ввода – проверка допустимости входных данных до их использования. Синтаксическая валидация проверяет формат и структуру, например корректность записи даты; семантическая – допустимость значения в предметном контексте, например порядок начала и окончания периода. Не заменяет авторизацию и разделение данных и команд.

СИНТАКСИЧЕСКАЯ ВАЛИДАЦИЯ

Правильно ли значение устроено?

формат, тип, длина, структура: дата записана в ожидаемом виде, код состоит из буквы и цифры

СЕМАНТИЧЕСКАЯ ВАЛИДАЦИЯ

Допустимо ли оно для этой операции?

предметный контекст: дата подходит для визита, тип документа и аудитория известны сервису

Форма – условие приёма;
предметные ограничения определяют допустимость

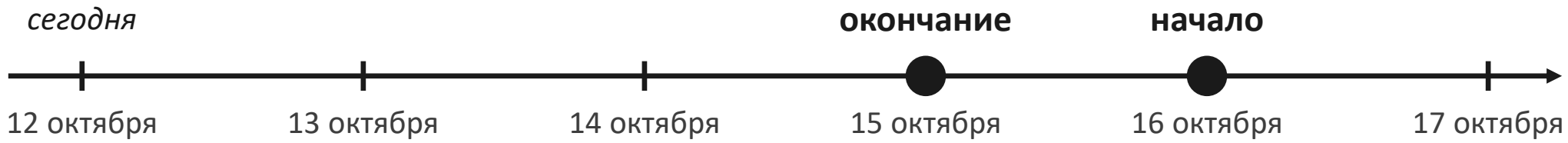
Рабочее определение курса

Дата визита: форма и смысл

СЕГОДНЯ
`2026-10-12`

НАЧАЛО ПЕРИОДА
`2026-10-16`
формат верен · дата в будущем

ОКОНЧАНИЕ ПЕРИОДА
`2026-10-15`
формат верен · дата в будущем



ПРАВИЛО: НАЧАЛО ≤ ОКОНЧАНИЕ
не выполнено – период отклонён

ДОПУСТИМЫЙ ПЕРИОД
`2026-10-15` → `2026-10-16`

Допустимость комбинации полей проверяется отдельно от их индивидуального формата

Тип и реквизиты документа

ПОЛЕ
тип документа = **X9**

→

ФОРМА КОДА
буква и цифра – подходит

→

СПРАВОЧНИК
P1, **P2** – кода **X9** в нём нет

→

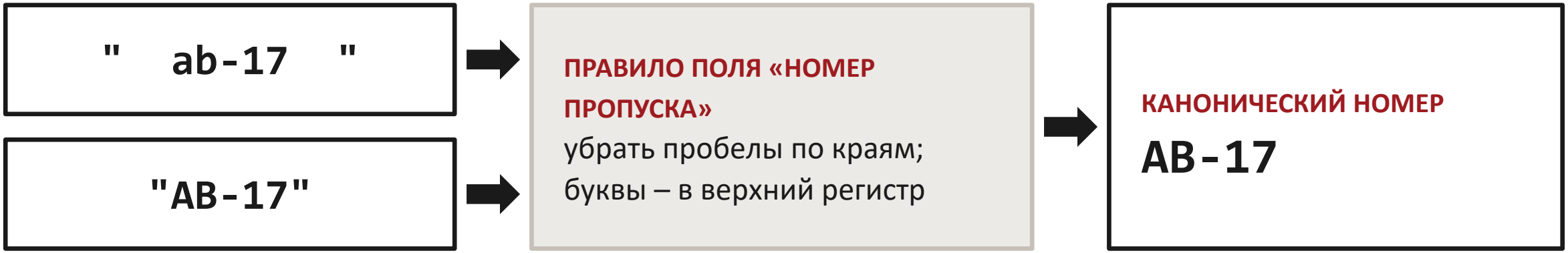
РЕШЕНИЕ
отклонить; «ближайший» код не подставлять

Коды P1, P2 и X9 условные. Реквизиты документа проверяются отдельно: у них свои структурные ограничения по учебному контракту

Для закрытого справочника допустимость определяется членством в нём

Одно значение – разные записи

Нормализация ввода – приведение данных к каноническому виду до проверки и использования, чтобы разные представления одного значения не обходили валидацию.



Считать эти записи одним значением можно, только если так установлено контрактом именно этого поля

Разные записи могут обозначать одно значение;
правила нормализации задаёт контракт поля

Рабочее определение курса

Порядок обработки входа

ОПЕРАТОР ВВОДИТ НОМЕР С БЛАНКА КАРТЫ
`"  ab-17  "`

В РЕЕСТРЕ УЖЕ ЕСТЬ ПРОПУСК
`AB-17`

НЕПРАВИЛЬНО: проверка до нормализации

проверка дубля по сырой строке `"  ab-17  "` – совпадений нет

↓

сохранение обрезает пробелы и поднимает регистр – **AB-17**

↓

в реестре второй пропуск **AB-17**: дубль сохранён

ИСПРАВЛЕНО: проверка после нормализации

ограничения размера и структуры, однозначный разбор

↓

нормализация: `"  ab-17  "` → **AB-17**

↓

проверка дубля по **AB-17** – найден, запрос отклонён

ИНВАРИАНТ ХРАНИЛИЩА уникальное ограничение по сохраняемому каноническому значению

Проверяем то представление, которое используется и сохраняется

Разрешить известное или запретить?

ВВОД
тип документа = **Р3**

РАЗРЕШАЮЩЕЕ ПРАВИЛО
тип ∈ { **Р1**, **Р2** }

Р3 ОТКЛОНЁН
кода нет среди разрешённых

ЗАПРЕТИТЕЛЬНЫЙ ФИЛЬТР
отклонять строки со знаками **%** и **!**

Р3 ОШИБОЧНО ПРИНЯТ
запрещённых знаков нет, но и код не разрешён

Закрытый справочник: перечисляем разрешённые значения.
Свободный текст: символы не вырезаем,
безопасность обеспечиваем в месте использования



Проверка формы не завершает валидацию

ВВОД
номер аудитории = **999**

ПРОВЕРКА ФОРМЫ
только цифры – пройдена

Какое одно предметное условие нужно проверить дополнительно?

ДОПУСТИМЫЙ ОТВЕТ
аудитория существует в справочнике помещений

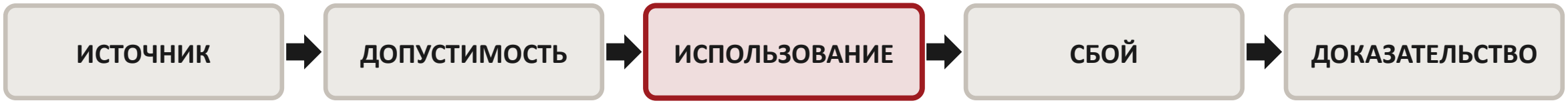
ДОПУСТИМЫЙ ОТВЕТ
помещение разрешено для оформления визита – это свойство помещения, а не право пользователя

Форма значения не отменяет проверки его смысла в данной операции

Секция 3

Данные, команды и файловые пути

Корректная строка опасна, если может изменить структуру операции



Когда данные меняют команду

Инъекция – класс дефектов, при которых недоверенный ввод становится частью команды, запроса или выражения и меняет его смысл. Корневая причина – нарушение разделения данных и команд.

СТРУКТУРА
постоянная часть команды

ДАННЫЕ
недоверенное значение извне

→

ОДНА СТРОКА
граница между структурой и данными исчезла

→

ИНТЕРПРЕТАТОР
разбирает всю строку как команду: значение может стать синтаксисом

Кто определяет структуру операции – программа или её вход?

**Значение должно менять искомое совпадение,
а не само условие поиска**

Рабочее определение курса

Поиск заявок: опасная сборка

Оператор ищет заявки по точному имени и дате

учебная СУБД: SQLite

ВВОД ОПЕРАТОРА

```
visit_date = "2026-10-15"
input_name = "Иванова"
```

ТАБЛИЦА visit_requests · УЧЕБНЫЙ НАБОР

```
VIS-21 | Иванова  | 2026-10-15
VIS-22 | Петров   | 2026-10-15
VIS-23 | Сидорова | 2026-10-16
```

СБОРКА ЗАПРОСА · ВЕРСИЯ 1.5

```
query = "SELECT id, guest_name, visit_date FROM visit_requests "
      + "WHERE visit_date = '" + visit_date + "' "
      + "AND guest_name = '" + input_name + "'"
```

РЕЗУЛЬТАТ `VIS-21 | Иванова | 2026-10-15`

В каких местах входные значения становятся частью текста запроса?

Вход, изменивший условие

ДАТА – БЕЗ ИЗМЕНЕНИЙ

```
visit_date = "2026-10-15"
```

ИМЯ – ТОЧНОЕ ЗНАЧЕНИЕ ВВОДА

```
' OR 1=1 --
```

ИТОГОВАЯ КОМАНДА · SQLite

перенос перед WHERE – только для показа

```
SELECT id, guest_name, visit_date FROM visit_requests
WHERE visit_date = '2026-10-15' AND guest_name = '' OR 1=1 --'
```

ЛОГИЧЕСКИЙ СМЫСЛ УСЛОВИЯ

скобки – пояснение, в запросе их нет

```
(visit_date = '2026-10-15' AND guest_name = '') OR 1=1
```

' закрывает литерал · **OR 1=1** истинно всегда · **--** комментирует остаток строки · **AND** связывает сильнее **OR**

РЕЗУЛЬТАТ **VIS-21**, **VIS-22**, **VIS-23** – три записи, включая заявку на 16 октября

Ввод изменил структуру условия, а не просто значение имени

Параметризация сохраняет структуру

Разделение данных и команд – построение обращения к интерпретатору так, чтобы входные значения обрабатывались как данные и не могли изменить структуру команды или запроса. Например, значения передаются параметрами SQL-запроса; проверка формата ввода сама по себе такого разделения не обеспечивает.

ВЕРСИЯ 1.6 · ПСЕВДОКОД, ИМЕНОВАННЫЕ ПАРАМЕТРЫ SQLite

```
sql = "SELECT id, guest_name, visit_date FROM visit_requests "
    + "WHERE visit_date = :visit_date AND guest_name = :guest_name"
db.execute(sql, { "visit_date": visit_date, "guest_name": input_name })
```

ТОТ ЖЕ ВВОД `' OR 1=1 --` связывается как одно значение `:guest_name` – 0 записей

конкатенация, версия 1.5 – 3 записи

параметры, версия 1.6 – 0 записей

Параметризация сохраняет структуру запроса;
допустимость ввода – отдельная задача валидации

Рабочее определение курса

Почему фильтр символов не спасает

Вопрос экрана 4: оператор ищет законную фамилию O'Брайен

Конкатенация

`guest_name = 'O'Брайен'`

апостроф закрыл литерал раньше времени; атаки нет, а запрос не разобран

ОШИБКА ЗАПРОСА

Запрет апострофа

O'Брайен – отклонён; **OБрайен** – другое имя

и запрет, и вырезание символа портят предметные данные

ЗАКОННЫЙ ВВОД СЛОМАН

Параметризация

`:guest_name` = **O'Брайен**

одно литеральное значение; такой фамилии в учебном наборе нет

ОШИБКИ НЕТ: 0 СОВПАДЕНИЙ

Для свободного текста структурное разделение сохраняет данные и защищает запрос; запрет подозрительных символов его не заменяет



Та же проблема в другом интерпретаторе

ИМЯ ФАЙЛА ОТЧЁТА ПРИХОДИТ ИЗВНЕ

```
report_name = "visitors.csv; echo CHECK"
```

ОДНА СТРОКА ДЛЯ ОБОЛОЧКИ

«;» отделяет вторую команду

```
shell("report-tool --name " + report_name)
оболочка выполнит:  report-tool --name visitors.csv ; echo CHECK
```

ПРОГРАММА И СПИСОК АРГУМЕНТОВ, БЕЗ ОБОЛОЧКИ

вся строка – один аргумент

```
run(["report-tool", "--name", report_name], shell=False)
программа получит:  --name  "visitors.csv; echo CHECK"
```

shell и run – псевдокод, а не интерфейс конкретного языка; echo CHECK – безобидная демонстрационная команда

Структурированный вызов сохраняет границу команды и данных; допустимость аргумента для программы проверяется отдельно

Имя файла – недоверенное значение

КАТАЛОГ ВЫГРУЗКИ СПИСКОВ

```
/srv/guestpass/
└── exports/            разрешённый корень
    └── visitors-2026-10-15.csv
```

ОБЫЧНОЕ ИМЯ В ЗАПРОСЕ

```
visitors-2026-10-15.csv
```

ПРИШЛО ВМЕСТО ИМЕНИ

```
../drafts/summary.csv
```

УСЛОВНАЯ ЛОГИКА ВЫДАЧИ

проверяется только существование файла

```
target = join(export_root, input_file)
if exists(target): send(target)
```

Какой файл фактически выберет сервис после объединения путей?

Канонический путь: где файл?

Канонический путь – однозначное итоговое расположение ресурса после разрешения относительных элементов, разделителей, символических ссылок и других допустимых вариантов записи пути. Канонизация не является самостоятельной защитой: принадлежность итогового пути разрешённому корню проверяется отдельно.

КОРЕНЬ	`/srv/guestpass/exports`
ИМЯ	`../drafts/summary.csv`
ОБЪЕДИНЕНО	`/srv/guestpass/exports/../drafts/summary.csv`
ИТОГ	`/srv/guestpass/drafts/summary.csv`
ВЕРДИКТ	**ВНЕ РАЗРЕШЁННОГО КОРНЯ**

УЧЕБНАЯ СХЕМА, БЕЗ ССЫЛОК

```
/srv/guestpass/
├── exports/   корень
│   └── visitors-….csv
└── drafts/
    └── summary.csv
```

Канонизация показывает итоговое расположение; разрешённость проверяется отдельно

Рабочее определение курса



Контроль границы разрешённого каталога

РАЗРЕШЁННЫЙ КОРЕНЬ ПОСЛЕ КАНОНИЗАЦИИ
`/srv/guestpass/exports`

ОБЫЧНОЕ ИМЯ – ВНУТРИ КОРНЯ
`…/exports/visitors-2026-10-15.csv`

СРАВНЕНИЕ ТЕКСТОВОГО ПРЕФИКСА – НЕНАДЁЖНО
`/srv/guestpass/exports-old/archive.csv` начинается с `/srv/guestpass/exports`
проверка префикса пройдена, хотя exports-old – соседний каталог, а не вложенный

СРАВНЕНИЕ ПО КОМПОНЕНТАМ ПУТИ
`srv / guestpass / exports` и `srv / guestpass / exports-old` – третий компонент другой
допустим путь, который равен корню или лежит внутри него по границам компонентов

ВНЕ КОРНЯ
`../drafts/summary.csv` – отказ

ВНУТРИ КОРНЯ
`visitors-2026-10-15.csv` – файл выдан

Сначала итоговый ресурс, затем проверка разрешённой области



Два контекста, разные защиты

SQL-инъекция

ЧТО ПРОИСХОДИТ

ввод влияет на структуру запроса

ПРИЧИНА

смешение данных и команд

ИСПРАВЛЕНИЕ

параметризация: значения отдельно от структуры

Обход разрешённого каталога

ЧТО ПРОИСХОДИТ

путь после разрешения элементов указывает вне корня

ПРИЧИНА

канонический результат не проверяется

ИСПРАВЛЕНИЕ

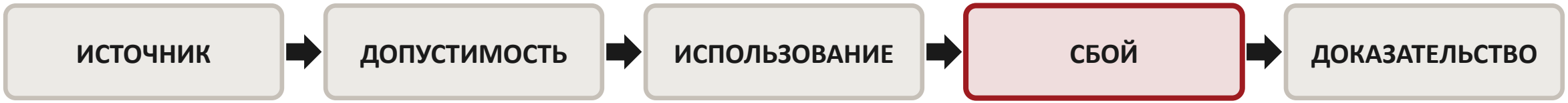
проверка принадлежности корню; по возможности – без пути от пользователя

Одна и та же недоверенная строка требует защиты, соответствующей месту её использования

Секция 4

Безопасная обработка ошибок

При сбое нельзя передавать внутреннюю диагностику внешнему адресату



Что раскрывает сырое исключение

КЛИЕНТ
корректная заявка, дата допустима

→

СЕРВИС
запрашивает внешнюю службу согласования

→

ОТВЕТ СЛУЖБЫ ПОВРЕЖДЁН
сбой при разборе, до обращения к реестру

СЫРОЙ ОТВЕТ КЛИЕНТУ *фиктивный пример*

```
500 Internal Server Error
ParseError in CreateVisitRequest
at /srv/guestpass/app/create_visit.py:117
Stack: request_handler -> parse_partner_response -> raise ParseError
```

РАСКРЫТО: тип исключения

РАСКРЫТО: путь к коду

РАСКРЫТО: структура вызовов

Сбой внешней зависимости нельзя исправить полем формы;
сырой ответ раскрывает внутреннее устройство приложения

Один сбой – два адресата

Безопасная обработка ошибок – обработка сбоя с предсказуемым безопасным результатом: клиент получает необходимую информацию без утечки внутренних деталей и секретов. Диагностический контекст сохраняется с контролем доступа и минимизацией чувствительных данных.

неожиданный ParseError при разборе ответа внешней службы

КЛИЕНТ

статус; безопасная формулировка – исправимая или общая; идентификатор

ИНЖЕНЕРНАЯ ДИАГНОСТИКА

тип, стадия, версия; контекст без секретов; доступ ограничен

Два сообщения относятся к одному событию, но содержат разные сведения

Рабочее определение курса



Клиенту, в журнал или никуда?

Только клиенту

- понятная причина исправимой ошибки

Только в журнал с контролем доступа

- тип внутреннего исключения
- стадия обработки

Ни в ответ, ни в журнал

- пароль, секрет
- полные реквизиты документа

допустимое хранение в предметных данных это не отменяет

В ОБА КАНАЛА

`incident_id` – связывает ответ клиенту с записью журнала

Содержание сообщения определяется назначением канала и минимизацией данных

Идентификатор связывает ответ и лог

Неожиданный сбой при разборе ответа внешней службы

ОТВЕТ КЛИЕНТУ — *без типа исключения, пути и стека*

```
{"error": "Не удалось обработать запрос", "incident_id": "INC-014"}
```

один идентификатор в двух каналах

ЗАПИСЬ В ЖУРНАЛЕ · ДОСТУП ОГРАНИЧЕН — *версия 1.4 относится только к этому эпизоду*

```
INC-014 | create_visit | parse_failed | version 1.4
```

Диагностика сохраняется
без раскрытия внутренних деталей клиенту



Три провала обработки исключений

Вернуть всё исключение

клиент видит тип исключения, путь к коду и стек вызовов

ИТОГ

раскрытие наружу

Поймать и промолчать

ответ выглядит безопаснее, но записи о сбое нет

ИТОГ

причину не восстановить

Скрыть снаружи, записать всё

в журнале – полный запрос с чувствительными полями

ИТОГ

утечка перенесена в журнал

ПРАВИЛЬНОЕ ПОВЕДЕНИЕ

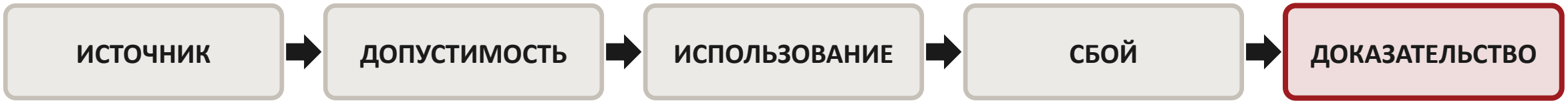
безопасный ответ + идентификатор инцидента + минимальная защищённая диагностика

Система должна сдерживать раскрытие и сохранять расследуемость

Секция 5

Доказанный цикл исправления

Защитный механизм подтверждён лишь в охваченных проверками условиях



Сценарий атаки и причина

Воспроизводимый сценарий	
ПРЕДУСЛОВИЯ	версия 1.5, SQLite; учебный набор из трёх записей; поиск собран конкатенацией
ДЕЙСТВИЯ	поиск: дата **2026-10-15**, имя **' OR 1=1 --**
НАБЛЮДАЕМЫЙ РЕЗУЛЬТАТ	**VIS-21**, **VIS-22**, **VIS-23** – три записи
ОЖИДАЕМЫЙ РЕЗУЛЬТАТ	0 совпадений: буквального имени **' OR 1=1 --** в наборе нет

ПРИЧИНА – МЕХАНИЗМ, А НЕ СИМПТОМ И НЕ НОМЕР СТРОКИ

input_name включается в текст SQL и меняет логику **WHERE**: нарушено разделение данных и команд

В доказательный цикл идут фиксированный вход, исходная версия и наблюдаемое нарушение

Проверки после параметризации

версия 1.5 → **ИЗМЕНЕНИЕ CHANGE-06**
дата и имя – параметрами; данные набора те же → **версия 1.6**

НЕГАТИВНАЯ ПРОВЕРКА

`' OR 1=1 --`, 15 октября
→ 0 записей

структура WHERE не изменилась

ПОЗИТИВНАЯ ПРОВЕРКА

Иванова, 15 октября
→ **VIS-21**

штатный поиск работает

РЕГРЕССИОННАЯ ПРОВЕРКА

Сидорова, 15 октября → 0 записей
Сидорова, 16 октября → **VIS-23**

фильтр по дате работал и в версии 1.5

Исправление подтверждено для атаки, штатного поиска и затронутой фильтрации – на заданных данных, на версии 1.6

Инъекция: полная цепочка

ПРОБЛЕМА + ВОСПРОИЗВОДИМЫЙ СЦЕНАРИЙ
версия 1.5 · SQLite

СЦЕНАРИЙ дата **2026-10-15**, имя **' OR 1=1 --**
НАБЛЮДАЕТСЯ три записи: **VIS-21**, **VIS-22**, **VIS-23**
ОЖИДАЕТСЯ 0 записей: такого имени в наборе нет

ПРИЧИНА

Два входных значения конкатенируются в текст SQL – ввод меняет структуру WHERE

ИСПРАВЛЯЮЩЕЕ ИЗМЕНЕНИЕ
изменение CHANGE-06
версия 1.5 → 1.6

Дата и имя передаются параметрами **:visit_date** и **:guest_name**

ПРОВЕРКИ ИСПРАВЛЕННОЙ ВЕРСИИ + ФАКТИЧЕСКИЕ РЕЗУЛЬТАТЫ
версия 1.6

НЕГАТИВНАЯ **attack_literal**: **' OR 1=1 --**, 15 октября → 0 записей
ПОЗИТИВНАЯ **valid_name**: Иванова, 15 октября → **VIS-21**
РЕГРЕССИОННАЯ **date_filter**: Сидорова, 15 октября → 0 записей
Сидорова, 16 октября → **VIS-23**

Изменение устранило показанный механизм на определённой версии и в проверенном сценарии

Перенос метода на новое условие

ЖАЛОБА

оператор запрашивает список **visitors-2026-10-17.csv**, которого в каталоге нет

ОТВЕТ СЕРВИСА

`FileNotFoundError`
`/srv/guestpass/exports/visitors-2026-10-17.csv`

БЕЗОПАСНЫЙ КОНТРАКТ

нет списка – ответ 404 «Файл не найден», без исключения и внутреннего пути; существующий список скачивается как прежде

НЕГАТИВНАЯ ПРОВЕРКА

нет файла **…-10-17.csv** – статус 404 и безопасное сообщение, без **FileNotFoundError** и без **/srv/guestpass/…**

ПОЗИТИВНАЯ ПРОВЕРКА

есть файл **visitors-2026-10-15.csv** – выгрузка возвращает подготовленный CSV без повреждения содержимого

Исправление обработки ошибки проверяем вместе с сохранением нормальной функции

Выводы и продолжение курса

1. **Где проверять:** точки входа следуют из потоков и границ доверия; важно последующее использование
2. **Что проверять:** форму, предметную допустимость и каноническое представление; для путей – разрешённый корень
3. **Как не менять структуру команд:** значения передаются отдельно от команды, в SQL – параметрами
4. **Что сообщать при сбое:** безопасный внешний ответ и минимальная защищённая диагностика – разные каналы
5. **Как подтвердить исправление:** негативная, позитивная и регрессионная роли, фактические результаты и версия

СЕМИНАР 06
самостоятельная работа на отдельном учебном API

КР
применение материала лекции на новой ситуации

ЛЕКЦИЯ 07
аутентификация и контроль доступа


ВЫСШАЯ • ШКОЛА • ЭКОНОМИКИ •